

COMPILER DESIGN

PROGRAMMING EXERCISE 3

MAANYA BHARDWAJ
PES2UG23CS311

CODE:

symbol_table.h

```
#ifndef SYMBOL_TABLE_H
#define SYMBOL_TABLE_H

#define MAX 200

typedef struct Symbol {
    char name[50];
    char kind[30];
    char type[100];
    char storage[20];
    int size;
    char scope[20];
    int line;
    int column;
    char file[50];
    char value[50];
} Symbol;

void insert(char*, char*, char*, char*, int, char*, int, int, char*);
void display();
int get_size(char*);
int lookup(char*);

#endif
```

symbol_table.c

```
#include <stdio.h>
#include <string.h>
#include "symbol_table.h"

Symbol table[MAX];
int count = 0;

int lookup(char* name) {
    for(int i=0;i<count;i++) {
        if(strcmp(table[i].name, name)==0)
            return i;
    }
    return -1;
}

void insert(char* name, char* kind, char* type, char* storage,
            int size, char* scope, int line, int column, char* file) {

    if(lookup(name) != -1) return;

    strcpy(table[count].name, name);
    strcpy(table[count].kind, kind);
    strcpy(table[count].type, type);
    strcpy(table[count].storage, storage);
    table[count].size = size;
    strcpy(table[count].scope, scope);
    table[count].line = line;
    table[count].column = column;
    strcpy(table[count].file, file);
    strcpy(table[count].value, "NULL");

    count++;
}

int get_size(char* type) {
    if(strstr(type, "*")) return 8; // pointer
    if(strstr(type, "int")) return 4;
    if(strstr(type, "float")) return 4;
}
```

```

    if(strstr(type, "char")) return 1;
    return 0;
}

void display() {
    printf("\nSYMBOL TABLE:\n");

    printf("-----\n");
    printf("Name\tKind\t\tType\t\tStorage\tSize\tScope\tLine\tCol\tValue\n");
    printf("-----\n");

    for(int i=0;i<count;i++) {
        printf("%s\t%s\t%s\t\t%s\t\t%d\t\t%d\t\t%d\t\t\t\n",
            table[i].name,
            table[i].kind,
            table[i].type,
            table[i].storage,
            table[i].size,
            table[i].scope,
            table[i].line,
            table[i].column,
            table[i].value
        );
    }
}

```

lexer.l

```

#include "parser.tab.h"
#include <string.h>
int line=1, column=1;
%}

%%

```

```

"int"    { return INT; }
"float"  { return FLOAT; }
"char"   { return CHAR; }

"static" { return STATIC; }
"extern" { return EXTERN; }
"register" { return REGISTER; }

"typedef" { return TYPEDEF; }
"struct"  { return STRUCT; }
"union"   { return UNION; }
"enum"    { return ENUM; }

[a-zA-Z_][a-zA-Z0-9_]* { yylval.str=strdup(yytext); return ID; }

[0-9]+    { return NUM; }

"*"       { return STAR; }
"["       { return LBRACKET; }
"]"       { return RBRACKET; }

"("       { return LPAREN; }
")"       { return RPAREN; }
"{"       { return LBACE; }
"}"       { return RBACE; }

";"       { return SEMICOLON; }
","       { return COMMA; }

[ \t]     { column++; }
\n        { line++; column=1; }

.         { return yytext[0]; }

%%

```

```
int yywrap(){ return 1; }
```

parser.y

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "symbol_table.h"

extern int line, column;

char current_type[100];
char current_scope[20] = "global";

void yyerror(const char *s);
int yylex();
%}

%union { char* str; }

%token <str> ID
%token INT FLOAT CHAR STATIC EXTERN REGISTER TYPEDEF STRUCT UNION ENUM
%token NUM
%token STAR LBRACKET RBRACKET
%token LPAREN RPAREN LBRACE RBRACE SEMICOLON COMMA

%%

program:
    program stmt
    |
    ;

stmt:
    declaration
```

```

| function
| typedef_stmt
| struct_stmt
| union_stmt
| enum_stmt
;

// ----- DECLARATIONS -----

// normal
declaration:
    type ID SEMICOLON {
        insert($2,"variable",current_type,"auto",
            get_size(current_type),current_scope,line,column,"input.c");
    }

// static
| STATIC type ID SEMICOLON {
    insert($3,"variable",current_type,"static",
        get_size(current_type),current_scope,line,column,"input.c");
}

// extern
| EXTERN type ID SEMICOLON {
    insert($3,"variable",current_type,"extern",
        get_size(current_type),current_scope,line,column,"input.c");
}

// register
| REGISTER type ID SEMICOLON {
    insert($3,"variable",current_type,"register",
        get_size(current_type),current_scope,line,column,"input.c");
}

// pointer
| type STAR ID SEMICOLON {
    char t[100]; sprintf(t,"%s*",current_type);
    insert($3,"variable",t,"auto",8,current_scope,line,column,"input.c");
}

// array
| type ID LBRACKET NUM RBRACKET SEMICOLON {

```

```

        char t[100]; sprintf(t,"%s[]",current_type);
        insert($2,"variable",t,"auto",
        get_size(current_type)*10,current_scope,line,column,"input.c");
    }
;

type:
    INT { strcpy(current_type,"int"); }
    | FLOAT { strcpy(current_type,"float"); }
    | CHAR { strcpy(current_type,"char"); }
;

// ----- FUNCTION -----

function:
    type ID LPAREN params RPAREN block {
        insert($2,"function",current_type,"auto",0,"global",line,column,"input.c");
    }
;

params:
    param_list
    |
;

param_list:
    param
    | param_list COMMA param
;

param:
    type ID {
        insert($2,"parameter",current_type,"auto",
        get_size(current_type),"function",line,column,"input.c");
    }
;

// ----- BLOCK -----

block:
    LBRACE { strcpy(current_scope,"block"); }
    stmt_list

```

```

    RBRACE { strcpy(current_scope,"global"); }
    ;

stmt_list:
    stmt_list stmt
    |
    ;

// ----- TYPEDEF -----

typedef_stmt:
    TYPEDEF type ID SEMICOLON {
        insert($3,"typedef",current_type,"auto",
            get_size(current_type),"global",line,column,"input.c");
    }
    ;

// ----- STRUCT -----

struct_stmt:
    STRUCT ID LBRACE RBRACE SEMICOLON {
        insert($2,"struct-tag","struct","auto",0,"global",line,column,"input.c");
    }
    ;

// ----- UNION -----

union_stmt:
    UNION ID LBRACE RBRACE SEMICOLON {
        insert($2,"union-tag","union","auto",0,"global",line,column,"input.c");
    }
    ;

// ----- ENUM -----

enum_stmt:
    ENUM ID LBRACE enum_list RBRACE SEMICOLON {
        insert($2,"enum-tag","enum","auto",4,"global",line,column,"input.c");
    }
    ;

enum_list:

```



```

    ID {
        insert($1,"enum-const","int","auto",4,"global",line,column,"input.c");
    }
| enum_list COMMA ID {
    insert($3,"enum-const","int","auto",4,"global",line,column,"input.c");
}
;

%%

void yyerror(const char *s){
    printf("Error: %s at line %d\n",s,line);
}

int main(){
    yyparse();
    display();
    return 0;
}

```

Input.c

```

int a;
float b;
static int x;
extern char c;
register int r;

typedef int myInt;

struct Person { };
union Data { };

enum color { RED, GREEN, BLUE };

int add(int x, int y) {
    int z;
}

```

Output :

```
add      function      int      auto      0      global 10      1      NULL
maanyabhardwaj@Maanyabhardwaj PE3 % flex lexer.1
bison -d parser.y
gcc lex.yy.c parser.tab.c symbol_table.c -o symtab
[./symtab < input.c
```

SYMBOL TABLE:

Name	Kind	Type	Storage	Size	Scope	Line	Col	Value
a	variable	int	auto	4	global	1	2	NULL
b	variable	float	auto	4	global	2	2	NULL
x	variable	int	static	4	global	3	3	NULL
c	variable	char	extern	1	global	4	3	NULL
r	variable	int	register	4	global	5	3	NULL
myInt	typedef int	auto	4	global	7	3	NULL	
Person	struct-tag	struct	auto	0	global	9	4	NULL
Data	union-tag	union	auto	0	global	10	4	NULL
RED	enum-const	int	auto	4	global	12	4	NULL
GREEN	enum-const	int	auto	4	global	12	5	NULL
BLUE	enum-const	int	auto	4	global	12	6	NULL
color	enum-tag	enum	auto	4	global	12	7	NULL
y	parameter	int	auto	4	function	14	5	NULL
z	variable	int	auto	4	block	15	6	NULL
add	function	int	auto	0	global	16	1	NULL

```
maanyabhardwaj@Maanyabhardwaj PE3 %
```